

UNIVERSIDAD ESTATAL PENÍNSULA DE SANTA ELENA

FACULTAD DE SISTEMAS Y TELECOMUNICACIONES

CARRERA DE INGENIERÍA EN SOFTWARE

ASIGNATURA:

SISTEMAS DISTRIBUIDOS Y CLOUD COMPUTING

TEMA:

ENTREGA FINAL — AUDITORÍA CRUZADA (2026)

Expediente Final · Red Team + Blue Team

ELABORADO POR (GRUPO 05 — Wyrdly):

Andy Bryan Alejandro Vera
Alisson Yamel Reyes Ricardo
Gino Maximiliano Bermúdez Santos
Yandris Miguel Rivera Torres

CURSO Y PARALELO: SOFTWARE 7/1

DOCENTE: ING. ANTHONY ABRAHAN PACHAY ESPINOZA

LA LIBERTAD – ECUADOR

Octubre 2026

Índice

1. Parte A — Evidencia como Red Team **2**
1.1. Identificación del equipo auditado . . . 2
1.2. Reproducibilidad . . . 2
1.3. Hallazgos emitidos . . . 2
1.3.1. Resumen cuantitativo . . . 2
1.3.2. Tabla de GitHub Issues (Issues #112–#120) . . . 3
1.4. Resultado posterior a la réplica del Blue Team auditado . . . 4
1.5. Top 3 hallazgos más relevantes . . . 5

2. Parte B — Evidencia como Blue Team **5**
2.1. Identificación del repositorio propio . . . 6
2.2. Issues recibidos y respuestas . . . 6
2.2.1. Resumen cuantitativo . . . 6
2.2.2. Tabla de respuestas por hallazgo . . . 6

3. Reflexión Técnica **7**

4. Evidencias **7**
4.1. Como Red Team . . . 7
4.2. Como Blue Team . . . 8

Anexo A — Detalle de Hallazgos Emitidos **9**

Anexo B — Síntesis de Réplicas como Blue Team **11**

Asignatura: Sistemas Distribuidos y Cloud Computing · **Curso:** Software 7/1
Docente: ING. ANTHONY ABRAHAN PACHAY ESPINOZA
Grupo: 05 — Wyrdly (red social distribuida en Quarkus 3 · React 19 · Neo4j · Redis · RustFS)

Rol Red Team: auditor del Grupo 03 **Rol Blue Team:** auditado por el Grupo 02
Hallazgos emitidos (GitHub Issues): 9 **Hallazgos recibidos:** 7

1 Parte A — Evidencia como Red Team

1.1 Identificación del equipo auditado

Equipo auditado:	Grupo 03 — *red-social-distribuida*
Tecnologías:	Frontend React 19 · Backend Quarkus 3.33 LTS (Java 21) · Neo4j 5 · Redis 7 · MinIO
Repositorio auditado:	https://github.com/Joseepc18/red-social-distribuida.git
Rama auditada:	`main`
SHA del commit congelado:	`454d3f26141f279f1babb349858506dc679aef83`
Hilo de auditoría en Moodle:	https://eva.upse.edu.ec/mod/forum/discuss.php?d=47712
GitHub Issues generados:	https://github.com/Joseepc18/red-social-distribuida/issues

1.2 Reproducibilidad

Reproducible **Reproducible (From Zero)**

El entorno se levantó con éxito ejecutando `gitclone` + `dockercomposeup-d--build` sin asistencia del equipo propietario. El primer arranque requirió tiempo adicional porque el servicio MinIO se compilaba desde código fuente dentro del build de Docker. Los arranques subsecuentes fueron fluidos. La documentación (`README.md`) fue la mejor observada en la cohorte: explica arquitectura, variables de entorno, modelo de grafo Cypher, endpoints, respaldo y restauración.

Observación: El tiempo de compilación inicial no estaba advertido en el `README.md`; se documentó como hallazgo H-01 (severidad Media).

1.3 Hallazgos emitidos

1.3.1 Resumen cuantitativo

Total (fase 1)	Crítica	Alta	Media	Baja	Obs.
9	1	8	0	0	0

Distribución final (post-contrarréplica)

Total	Crítica	Alta	Media	Baja	Retirados
9	0	2	4	3	0

GitHub Issues #112–#120 en https://github.com/Joseepc18/red-social-distribuida/issues

1.3.2 Tabla de GitHub Issues (Issues #112–#120)

ID	Issue	Sev. inicial	Sev. final	Hallazgo
H-10	#112	Crítica	Media	Clave privada RSA con `chmod644` en `docker-compose.yml:102`: el volumen `app_keys` expone `privateKey.pem` y `vapid-private.txt` con permisos world-readable. Permite falsificar JWT y suplantar el servidor VAPID si hay lectura arbitraria de archivos.
H-11	#113	Alta	Alta	Timing attack y enumeración de usuarios en login: rama «usuario inexistente» retorna en 5–15 ms; rama «contraseña incorrecta» tarda 80–120 ms por `BcryptUtil.matches`. Diferencia de ∼5× demostrada por ambos equipos.
H-12	#114	Alta	Media	Ausencia de `Content-Security-Policy` en `frontend/nginx.conf`: sesión JWT almacenada en `localStorage` queda expuesta ante cualquier XSS futuro; no existe barrera de navegador.
H-28	#115	Alta	Alta	Credencial inmutable: sin endpoint de cambio de contraseña ni lista negra de tokens. Cierre de sesión solo purga `localStorage`; el JWT sigue válido 24h y la contraseña no puede cambiarse.
H-05	#117	Alta	Media	`PostService` con 5 dependencias directas: `create()` orquesta validación, upload S3, commit Neo4j, evento CDI y ensamblado en 46 líneas con 4 bloques `try/catch` de compensación.
H-07	#116	Media	Baja	`AuthService` depende de `UserService` en lugar de `UserRepository`: acoplamiento transitivo sin impacto funcional. Degradado a Baja.
H-09	#118	Media	Baja	Mezcla de idiomas: paquetes en español (`comentarios`, `notificaciones`) + métodos en inglés (`create`, `thread`) + tipo duplicado `UsuarioResumen`/`UserSummary`. Aceptado y degradado a Baja.
H-22	#119	Alta	Baja	WSOD en comentarios: `CommentThread.tsx` accede a `comment.autor.id` sin `?`; sin `<ErrorBoundary>` un nulo desmonta toda la app. G03 demostró que `MATCH(autor:Usuario)` excluye nulos. Degradado a Baja.
H-24	#120	Alta	Media	Sin timeout ni Circuit Breaker hacia Neo4j: las consultas bloquean el pool de workers hasta que Nginx corta a los 60 s (valor por defecto de `proxy_read_timeout`); `/q/health/live` sigue en 200 aunque el servicio sea inoperable.

1.4 Resultado posterior a la réplica del Blue Team auditado

ID	Hallazgo	Réplica G03	Estado final	Síntesis de la contrarréplica
H-11	Timing attack en login	Aceptado	**Mantenido** **Alta**	G03 aportó medición propia (22 ms vs 116 ms) que confirma el hallazgo. Rate limit de Nginx no impide enumeración distribuida por IP.
H-28	Sin cambio de clave ni revocación	Acept. c/obs.	**Mantenido** **Alta**	Corregimos ruta (`/me` no `/yo`) y retiramos «secuestro definitivo» del token; la falla real persiste: contraseña sin vía de cambio.
H-09	Mezcla de idiomas y tipo duplicado	Aceptado	**Mantenido** **Baja**	Unificamos severidad (era Media en el foro, Baja en el Issue).
H-10	Clave privada JWT `chmod644`	Acept. c/obs.	**Modificado** **Media**	Aceptamos: solo los backends montan `app_keys`; UID correcto es 185, no 1001. Mantenemos Media porque el mismo chmod afecta también `vapid-private.txt`.
H-12	Sin Content-Security-Policy	Acept. c/obs.	**Modificado** **Media**	Ningún XSS demostrado; no hay `dangerouslySetInnerHTML`. Mantenemos Media: JWT en `localStorage` queda sin barrera si aparece un XSS futuro.
H-24	Sin timeout/Circuit Breaker Neo4j	Acept. c/obs.	**Modificado** **Media**	Retiramos «caída en cascada»; no es Netty sino worker pool. Media: 60 s de degradación total sin señal al balanceador es un defecto de resiliencia medible.
H-05	`PostService` 5 dependencias	Acept. c/obs.	**Modificado** **Media**	`ImageValidator` tiene tests propios; hay cobertura de integración. Media: compensación de MinIO acoplada al mismo método de creación.
H-07	`AuthService` → `UserService`	Acept. c/obs.	**Modificado** **Baja**	Escenario 404 no reproducible en la versión congelada. Degradamos a Baja: acoplamiento transitivo sin impacto funcional observable.
H-22	WSOD en comentarios	Acept. c/obs.	**Modificado** **Baja**	G03 demostró que `MATCH(autor:Usuario)` excluye nulos. Sin camino reproducible para disparar el crash = Baja.

Resumen post-contrarréplica (Red Team G05 → Blue Team G03): **Mantenidos:** 3 (H-11 Alta, H-28 Alta, H-09 Baja) **Modificados:** 6 (H-10 Media, H-12 Media, H-24 Media, H-05 Media, H-07 Baja, H-22 Baja) **Retirados:** 0 **Distribución final:** 0 Críticos · 2 Altos · 4 Medios · 3 Bajos

1.5 Top 3 hallazgos más relevantes

Top 1 — H-10 · Clave privada RSA con chmod 644 (Issue #112)

Crítica **CWE-732 / CWE-552 · Permisos incorrectos sobre material criptográfico**
En `docker-compose.yml:102`, el contenedor `keys-init` ejecuta `chmod644/keys/*` sobre la clave privada RSA (`privateKey.pem`) y la clave VAPID (`vapid-private.txt`). Ambas quedan `rw-r--r--`: legibles por cualquier usuario del sistema de archivos del contenedor. Quien logre leer `privateKey.pem` puede firmar tokens JWT arbitrarios para cualquier `subject` y asumir privilegios de cualquier usuario del sistema. Es el único hallazgo Crítico de la auditoría.
Evidencia:

```
docker compose exec backend-1 ls -la /keys
# -rw-r--r-- 1 root root 1675 privateKey.pem
# docker-compose.yml:102: chmod 644 /keys/*
```

Resolución post-réplica: Modificado a Media — el Blue Team demostró que solo los dos backends montan el volumen y que el UID correcto es 185. Corrección confirmada: `chown185:185/keys/privateKey.pem&&chmod600/keys/privateKey.pem`.
GitHub Issue: https://github.com/Joseepc18/red-social-distribuida/issues/112

Top 2 — H-11 · Timing Attack y Enumeración de Usuarios en Login (Issue #113)

Alta **CWE-208 / CWE-204 · Información filtrada por diferencias de tiempo**
En `AuthService.login()`, cuando el usuario no existe el `Optional` es vacío y la rama retorna sin ejecutar `BcryptUtil.matches` (∼5–15 ms). Cuando el usuario existe con contraseña incorrecta, `matches` consume el costo criptográfico de bcrypt (∼80–120 ms). Diferencia de ∼5× confirmada por medición independiente del Blue Team G03 (mediana 22.3 ms vs 116.1 ms). Aunque el rate limit de Nginx limita las peticiones por IP, basta distribuir el escaneo entre múltiples IPs para enumerar la base de usuarios completa. **Estado final: Mantenido Alta.**
Corrección: Computar siempre `BcryptUtil.matches` con un hash ficticio precargado cuando el usuario no exista.
GitHub Issue: https://github.com/Joseepc18/red-social-distribuida/issues/113

Top 3 — H-28 · Credencial Inmutable: Sin Cambio de Contraseña ni Revocación JWT (Issue #115)

Alta **CWE-620 / CWE-613 · Contraseña permanente + sesión no revocable**
`AuthResource` solo expone `POST/registro` y `POST/login`. No existe `PUT/auth/password` ni lista negra de tokens en Redis. El cierre de sesión solo ejecuta `setSession(null)` en el cliente (`AuthProvider.tsx:68-71`); el servidor no tiene ninguna señal. Si la contraseña se filtra, el atacante puede emitir tokens indefinidamente sin que la víctima pueda bloquearlo: la única salida es intervenir la base de datos manualmente. Para una red social con mensajería privada, esto constituye una cuenta comprometida sin vía de recuperación. **Estado final: Mantenido Alta.**
GitHub Issue: https://github.com/Joseepc18/red-social-distribuida/issues/115

2 Parte B — Evidencia como Blue Team

2.1 Identificación del repositorio propio

Proyecto:	Wyrdly — Red Social Distribuida
Repositorio:	https://github.com/yan2005dris-afk/wyrdly
Rama sometida a auditoría:	`develop`
SHA del commit congelado:	`8c062a6`
Red Team auditor:	Grupo 02
Hilo en Moodle:	https://eva.upse.edu.ec/mod/forum/discuss.php?d=47708
Issues recibidos:	https://github.com/yan2005dris-afk/wyrdly/issues (#173–#177, #184–#185)

2.2 Issues recibidos y respuestas

2.2.1 Resumen cuantitativo

Recibidos	Aceptado	Acept. c/Obs.	Impugnado
7	0	6	1

2.2.2 Tabla de respuestas por hallazgo

ID	Descripción	Nuestra respuesta	Resultado final (post-contrarréplica)
REP-001	Backend *unhealthy* en Windows por script JWT Bash	Acept. c/Obs.	Mantenido (Crítica) — El Red Team mantuvo la severidad por bloquear la reproducibilidad from-zero en Windows.
SEC-001	Rate Limiting «falso» — todas las peticiones devolvían 200	Impugnado	Retirado — El Red Team comprobó que ejecutó clics sobre el mismo post, activando el filtro de Idempotencia (Anti Rage-Click) en lugar del Rate Limiter.
REQ-002	Soporte de video roto — selector acepta `.mp4` pero backend rechaza	Acept. c/Obs.	Modificado (Baja) — Degradado a defecto UX por selector engañoso; imágenes con RustFS operan correctamente.
QUA-003	Ciclo `401Unauthorized` constante en notificaciones	Acept. c/Obs.	Modificado (Media) — El canal SSE no renueva token expirado; el backend opera correctamente.
UX-004	Botón «···» (opciones de post) sin acción — *dead click*	Acept. c/Obs.	Mantenido (Baja) — El Blue Team reconoce que exponer controles inoperativos incumple la sección 11 del enunciado.
RES-005	Colapso del frontend al detener Neo4j — mensajes contradictorios	Acept. c/Obs.	Modificado (Media) — Degradado: la falla radica en propagar `500` en lugar de `503ServiceUnavailable` semántico.
RES-006	Latencia en cascada al detener Redis (1.5–5.0 s/req) por timeout	Acept. c/Obs.	Mantenido (Media) — Ausencia de Circuit Breaker / Fail-fast confirmada.

Resumen post-contrarréplica del Red Team G02: **Mantenidos:** 3 (REP-001, UX-004, RES-006) **Modificados:** 3 (REQ-002, QUA-003, RES-005) **Retirados:** 1 (SEC-001)

3 Reflexión Técnica

Máximo 300 palabras. No se busca justificar errores, sino demostrar capacidad de análisis, criterio técnico y aprendizaje.

¿Qué hallazgo recibido considera técnicamente más importante y por qué?

El hallazgo REP-001 (backend *unhealthy* en Windows) es el más importante porque expone una inconsistencia de diseño sistémica: el proyecto ya automatizaba la generación de claves VAPID directamente en el `ENTRYPOINT` del contenedor, pero las claves JWT dependían de un script Bash externo al contenedor. Esta asimetría viola el principio de "configuración mínima" exigido en la rúbrica y convierte a Windows en un ciudadano de segunda clase para desplegar el sistema. Lo más relevante no es el fallo en sí, sino lo que revela: una decisión de arquitectura tomada de forma inconsistente entre módulos desarrollados en momentos distintos del sprint.

¿Qué aspecto del proyecto cambiaría en una nueva versión?

La estrategia de resiliencia de la capa de caché. Redis fue incorporado como optimización proactiva para el Rate Limiting y la Idempotencia de reacciones, lo cual es correcto. El error fue no añadir un Circuit Breaker con timeout fail-fast desde el inicio. En una nueva versión, todos los filtros que acceden a Redis incluirían `@CircuitBreaker` de `quarkus-smallrye-fault-tolerance` y un timeout de 200 ms, garantizando que la capa de optimización nunca degrade el endpoint principal cuando la caché no está disponible.

¿Qué problema no habían considerado durante el desarrollo?

La compatibilidad de los scripts de aprovisionamiento con entornos Windows nativos. Durante el sprint, todos los integrantes trabajamos en macOS y Linux, por lo que el vector "Windows sin Bash/WSL" nunca apareció en nuestros escenarios de prueba. La auditoría externa lo identificó de inmediato como un bloqueante crítico para la reproducibilidad. Este punto es el argumento más sólido a favor de incluir evaluadores con entornos heterogéneos desde las primeras iteraciones del proyecto.

4 Evidencias

Todos los artefactos listados a continuación deben permanecer accesibles durante el periodo de evaluación.

4.1 Como Red Team

1. **Documento de auditoría completo (H-01 a H-28):** `docs/AUDITORIARB/RedTeam/AUDITORIA.tex` y `AUDITORIA.pdf` en este repositorio.
2. **Matriz priorizada de Críticos y Altos (Issues #112–#120):** `docs/AUDITORIARB/RedTeam/AUDITORIA_CRITICOS_ALTOS.tex` y `AUDITORIA_CRITICOS_ALTOS.pdf`.
3. **Contrarréplica Red Team G05 → Blue Team G03 (Fase 3):** `docs/AUDITORIARB/RedTeam/contrarreplicaRedTeam.html`.
4. **GitHub Issues en repositorio auditado:** https://github.com/Joseepc18/red-social-distribuida/issues
5. **Hilo de auditoría en Moodle:** https://eva.upse.edu.ec/mod/forum/discuss.php?d=47712

4.2 Como Blue Team

1. **Repositorio propio (commit congelado `8c062a6`):** https://github.com/yan2005dris-afk/wyrdly/tree/8c062a6
2. **Issues recibidos (7):** #173 https://github.com/yan2005dris-afk/wyrdly/issues/173, #174 https://github.com/yan2005dris-afk/wyrdly/issues/174, #175 https://github.com/yan2005dris-afk/wyrdly/issues/175, #176 https://github.com/yan2005dris-afk/wyrdly/issues/176, #177 https://github.com/yan2005dris-afk/wyrdly/issues/177, #184 https://github.com/yan2005dris-afk/wyrdly/issues/184, #185 https://github.com/yan2005dris-afk/wyrdly/issues/185
3. **Réplicas oficiales en Issues + archivos locales:** `docs/AUDITORIARB/BlueTeam/Replica\{1-7\}.md`
4. **Contrarréplica del Red Team G02 (cierre):** `docs/AUDITORIARB/BlueTeam/ContraReplicaRedTeam.txt`
5. **Hilo en Moodle (Blue Team):** https://eva.upse.edu.ec/mod/forum/discuss.php?d=47708

Anexo A — Detalle de Hallazgos Emitidos (Issues #112–#120)

El texto completo de cada ficha técnica (pasos, PoC, evidencia exacta de código) está en `docs/AUDITORIARB/RedTeam/AUDITORIA_CRITICOS_ALTOS.tex`. Este anexo recoge la síntesis y el estado final tras la contrarréplica.

H-10 — Clave privada RSA `chmod 644` · Issue #112

Crítica → Media **CWE-732 · M4 Seguridad**

`docker-compose.yml:102`: `chmod644/keys/*` deja `privateKey.pem` y `vapid-private.txt` world-readable.

Contrarréplica: Aceptamos que solo los dos backends montan el volumen (UID 185, no 1001). Bajado a **Media** porque sin escape de contenedor o acceso al host la explotación directa no está demostrada; mantenemos porque el mismo fallo expone ambos secretos de firma del sistema.

Fix: `chown185:185/keys/privateKey.pem&&chmod600/keys/privateKey.pem`

H-11 — Timing Attack en Login · Issue #113

Alta → Alta **CWE-208 · M4 Seguridad**

Rama «no existe»: 5–15 ms. Rama «contraseña incorrecta»: 80–120 ms (bcrypt). G03 confirmó con medición propia: mediana 22.3 ms vs 116.1 ms (∼5×).

Fix: Precalcular siempre bcrypt con un hash ficticio cuando el Optional está vacío.

H-12 — Ausencia de Content-Security-Policy · Issue #114

Alta → Media **CWE-1021 / CWE-79 · M4 Seguridad**

Nginx no envía `Content-Security-Policy`; JWT en `localStorage`. Ningún XSS demostrado en el commit `454d3f2`.

Contrarréplica: Bajado a Media; mantenido porque `localStorage` queda sin barrera ante XSS futuro (dependencias, cambios de código).

H-28 — Credencial Inmutable · Issue #115

Alta → Alta **CWE-620 / CWE-613 · M4 Seguridad**

Sin `PUT/auth/password`; tokens no revocables (lifespan 86400 s); logout solo purga cliente.

Fix: Atributo `tokenVersion` en el nodo `:Usuario` de Neo4j como claim en el JWT; `PUT/api/auth/password` incrementa el contador.

H-05 — PostService Fat Service · Issue #117

Alta → Media **Diseño · M3 Calidad**

5 dependencias; `create()` 46 líneas; 4 bloques `try/catch` de compensación acoplados.

Fix: Descomponer en `PostCreator`, `PostImageAttacher`, `PostPersister`.

H-07 — AuthService → UserService · Issue #116

Media → Baja **Diseño · M3 Calidad**

Acoplamiento transitivo entre servicios. G03 demostró que registro devuelve 201 y que `profile()` delega directo a `UserRepository`. Degradado a Baja: sin impacto funcional.

H-09 — Mezcla de Idiomas y Tipo Duplicado · Issue #118

Media → Baja **Mantenibilidad · M3 Calidad**

Paquetes en español (`comentarios`, `notificaciones`) con métodos en inglés (`create`, `thread`, `save`). Tipo duplicado `UsuarioResumen`/`UserSummary` en uso simultáneo. El Blue Team lo aceptó; degradado a Baja por consenso.

Fix: Definir y documentar una convención de idioma única en toda la base de código.

H-22 — WSOD en Comentarios · Issue #119

Alta → Baja **CWE-754 · M5 UX / Robustez**

`CommentThread.tsx` accede a `comment.autor.id` sin `?`; sin `<ErrorBoundary>` un nulo desmonta toda la app.

Contrarréplica: Bajado a Baja — `MATCH(autor:Usuario)-[:COMENTA]->(c)` excluye nulos por diseño; camino de explotación no demostrado.

H-24 — Sin Timeout ni Circuit Breaker hacia Neo4j · Issue #120

Alta → Media **CWE-400 · M6 Resiliencia**

Peticiones bloqueadas hasta 60 s (valor por defecto de `proxy_read_timeout` de Nginx). `/q/health/live` en 200 aunque el servicio sea inoperable; balanceador no descarta réplicas.

Contrarréplica: Retiramos «caída en cascada». Media mantenida: degradación medible de 60 s/petición sin señal al balanceador.

Fix: `@Timeout(3s)` + `@CircuitBreaker` en repositorios Neo4j vía `quarkus-smallrye-fault-tolerance`.

Anexo B — Síntesis de Réplicas como Blue Team

REP-001 — ACEPTADO CON OBSERVACIONES

Se acepta la limitación. El script `generate-jwt-keys.sh` no es interpretado por PowerShell/CMD nativos. El directorio `jwt/` queda vacío, el backend no supera el healthcheck y entra en crash-loop. Solicitamos reclasificación a Media porque el fallo es de pre-configuración del host, no del código de aplicación. La base de código ya incluye `JwtKeyGenerator.java` (RSA puro en JDK) que podría integrarse en el `ENTRYPOINT`. *Contrarréplica Red Team: MANTENIDA (Crítica)* — un bloqueante total en Windows nativo no admite reclasificación aunque sea de pre-configuración.

SEC-001 — IMPUGNADO

El hallazgo describe una prueba metodológicamente incorrecta: 35 clics rápidos sobre el **mismo** post activan `ReactionIdempotencyFilter` (Anti Rage-Click), que devuelve `200OK` con `X-Cached-Response:true` sin consumir cuota de Rate Limit. Para agotar el Rate Limiter es necesario enviar peticiones a 31 recursos distintos. Se adjuntó script de reproducción y evidencia empírica del `HTTP429`. *Contrarréplica Red Team: RETIRADO.*

REQ-002 — ACEPTADO CON OBSERVACIONES

Se acepta el defecto. El atributo `accept="image/*,video/*"` en `CreatePostCard` permite seleccionar videos que el backend rechaza. El requisito base de RustFS se cumple íntegramente con imágenes. Severidad propuesta: Baja/UX. *Contrarréplica: MODIFICADO (Baja).*

QUA-003 — ACEPTADO CON OBSERVACIONES

Se acepta la falla raíz: `useNotificationStream` usa `fetch` directamente (no Axios), por lo que no renueva el token expirado y genera ciclos de `401`. El backend opera correctamente — rechaza tokens vencidos, como corresponde. *Contrarréplica: MODIFICADO (Media).*

UX-004 — ACEPTADO CON OBSERVACIONES

Se acepta el defecto. El botón `MoreHorizontal` en `PostCard.tsx` tiene estilos de hover pero sin manejador de evento. Viola la sección 11 del enunciado. Acción correctiva: eliminar el botón del `PostCard` hasta que la funcionalidad esté implementada. *Contrarréplica: MANTENIDO (Baja).*

RES-005 — ACEPTADO CON OBSERVACIONES

Se acepta la anomalía. Al caer Neo4j, Quarkus emite `500` sin mapper semántico; el frontend interpreta esto como expiración de sesión y purga `localStorage`, generando mensajes contradictorios. No hay corrupción de datos. *Contrarréplica: MODIFICADO (Media).*

RES-006 — ACEPTADO CON OBSERVACIONES

Se acepta la degradación de latencia (1.5–5.0 s por petición). Los filtros `ReactionIdempotencyFilter` y `ReactionRateLimitFilter` realizan llamadas síncronas a Redis con `quarkus.redis.timeout=2s`; sin Circuit Breaker, el timeout se acumula por filtro. *Contrarréplica: MANTENIDO (Media).*